

[AI-02] Báo Cáo Audit AI - Group 06

1. Thông Tin Nhóm

- Nhóm: Group 06 - KDBK
- Đề tài: T09 - Security Testing (DAST / SAST)
- Seminar track: Security testing với Semgrep/SAST, OWASP ZAP/DAST và AI-assisted triage.
- Thành viên:
 - Lê Trung Kiên - 23127075
 - Mai Thị Kim Duyên - 23127185
 - Lâm Hữu Khánh - 23127205
 - Lê Mai Hoài Bảo - 23127326
- Phạm vi audit: các nội dung AI hỗ trợ trong quá trình chọn công cụ, viết script Semgrep/ZAP, phân tích finding, tạo PoC/checklist kiểm chứng và biên tập report.
- Nguyên tắc đánh giá: AI output chỉ được xem là hợp lệ khi có human review và được đối chiếu với source code, output Semgrep/ZAP, unit test, request/response log, screenshot hoặc tài liệu chính thức.

2. Bảng Audit

Mục 1 - DAST, ZAP Và OWASP Top 10: Tìm Nguồn Và Tìm Hiểu Nền Tảng

1. Prompt + công cụ AI

- Công cụ: Gemini 3.1 Pro, NotebookLM cho Lê Trung Kiên.
- Prompt/nội dung yêu cầu:

Yêu cầu AI tìm nguồn chính thống và giải thích các khái niệm nền tảng về DAST, OWASP ZAP và OWASP Top 10 để phục vụ phần seminar Security Testing.

2. Nội dung AI hỗ trợ

AI hỗ trợ tổng hợp nội dung nền tảng:

- DAST là gì và khác SAST ở điểm nào.
- Vai trò của OWASP ZAP trong kiểm thử bảo mật động.
- Cách OWASP Top 10 được dùng để phân loại và ưu tiên rủi ro bảo mật.
- Các nguồn tham khảo nên dùng khi viết user guide, slide và report.

Lê Trung Kiên đã kiểm tra lại output AI trên nguồn chính thống của OWASP và ZAP trước khi đưa vào tài liệu.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

Nội dung AI được dùng ở mức hỗ trợ tìm hiểu và tổng hợp kiến thức. Sinh viên đã đối chiếu lại với tài liệu chính thống của OWASP và ZAP trước khi sử dụng.

4. Lý do theo ISTQB

- Nguồn tham chiếu phù hợp với mục tiêu học công cụ và chuẩn bảo mật.
- AI không được dùng làm nguồn bằng chứng cuối cùng; output được human review và cross-check với tài liệu chính thống.
- Nội dung thuộc dạng knowledge support, rủi ro thấp hơn so với kết luận defect/runtime evidence.
- Có traceability tới phần nội dung seminar về DAST, ZAP và OWASP Top 10.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã dùng phần đã kiểm tra để hỗ trợ biên soạn:

- Nội dung giới thiệu DAST/ZAP/OWASP Top 10 trong user guide và slide.
- Cách diễn giải vai trò của ZAP trong workflow kiểm thử bảo mật.
- Các liên kết/tài liệu tham khảo chính thống được ưu tiên hơn output AI.

Mục 2 - ZAP Scan Output Format: JSON Và HTML

1. Prompt + công cụ AI

- Công cụ: GPT-5.5 cho Lê Trung Kiên.
- Prompt/nội dung yêu cầu:

Yêu cầu AI hỗ trợ viết script cấu hình output format của ZAP scan ra JSON và HTML để phục vụ pipeline đọc kết quả và báo cáo thủ công.

2. Nội dung AI hỗ trợ

AI hỗ trợ phần script cho ZAP scan:

- Cho phép chọn report format `json` hoặc `html`.
- Tách đường dẫn output theo nhu cầu scan frontend/backend.
- Chuẩn hóa output để pipeline sau này có thể đọc JSON.
- Gợi ý cấu trúc phần tag trong report.

Lê Trung Kiên đã kiểm tra lại output, chỉnh sửa và bổ sung thêm phần tag để kết quả scan có thể trace với nhóm lỗi bảo mật liên quan.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

Script AI hỗ trợ đã được sinh viên kiểm tra lại, bổ sung tag và đối chiếu với nhu cầu xuất báo cáo JSON/HTML của workflow ZAP.

4. Lý do theo ISTQB

- Test objective rõ: ZAP scan phải xuất được định dạng phù hợp cho cả human review và pipeline xử lý sau.
- Có expected behavior cụ thể: chỉ chấp nhận output format phục vụ pipeline, đặc biệt là `json` và `html`.
- Sinh viên đã human review output và bổ sung metadata/tag cần thiết.
- Phù hợp regression testing cho CLI/report generation.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã kiểm tra và bổ sung:

- Cấu hình output report `json/html`.
- Tag lỗi trong output để hỗ trợ phân loại và đối chiếu.
- Đường dẫn output phù hợp với các mục tiêu scan khác nhau.

Mục 3 - ZAP GUI Scan Với Authentication: Firefox Localhost Config

1. Prompt + công cụ AI

- Công cụ: Gemini 3.1 Pro cho Lê Trung Kiên.
- Prompt/nội dung yêu cầu:

Yêu cầu AI giải thích và hướng dẫn trong quá trình tìm hiểu GUI Scan với authentication trên OWASP ZAP, đặc biệt khi cần cấu hình Firefox để cho phép kết nối localhost.

2. Nội dung AI hỗ trợ

Gemini hướng dẫn kiểm tra cấu hình Firefox `about:config` để allow localhost trong quá trình dùng ZAP GUI scan với authentication.

Lê Trung Kiên dùng hướng dẫn này như tài liệu hỗ trợ thao tác, sau đó kiểm tra lại trong quá trình thực hành GUI scan.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

Nội dung AI hỗ trợ đúng phạm vi hướng dẫn thao tác cấu hình môi trường local và đã được sinh viên kiểm tra lại khi thực hành.

4. Lý do theo ISTQB

- Đây là environment setup support, không phải bằng chứng defect cuối cùng.
- Sinh viên đã xác nhận lại bằng thao tác thực tế trong quá trình tìm hiểu GUI scan.
- Nội dung có precondition rõ: Firefox/ZAP GUI/proxy/local target.
- Rủi ro được kiểm soát vì chỉ áp dụng cho môi trường local seminar.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã dùng hướng dẫn này để hỗ trợ:

- Cấu hình trình duyệt khi scan ứng dụng local qua ZAP.
- Tìm hiểu authenticated scan bằng GUI.
- Ghi chú lại các điều kiện môi trường cần có khi chạy ZAP với localhost.

Mục 4 - ZAP AI Triage Và Testcase Generation

1. Prompt + công cụ AI

- Công cụ: GPT-5.5 cho Lê Trung Kiên.
- Prompt/nội dung yêu cầu:

Yêu cầu AI hỗ trợ viết script tạo AI triage report và testcase từ kết quả ZAP scan.

2. Nội dung AI hỗ trợ

AI hỗ trợ tạo script và output cho luồng ZAP AI triage:

- Đọc kết quả ZAP scan.
- Tạo nội dung AI triage theo từng alert/finding.
- Sinh testcase/checklist kiểm chứng.
- Chuẩn hóa các section cần có trong report.

Lê Trung Kiên đã kiểm tra lại code và output do AI tạo, xác nhận output có đủ các section yêu cầu, kiểm thử script và xác nhận output tốt.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

Script và output AI triage đã được sinh viên review, kiểm thử và xác nhận đáp ứng format/section yêu cầu.

4. Lý do theo ISTQB

- Có traceability từ input ZAP scan tới triage report và testcase.
- Có expected output cụ thể bằng các section bắt buộc trong report.
- Sinh viên đã kiểm tra code và output, không dùng raw AI output làm kết luận cuối.

- Phù hợp component/regression testing cho parser, renderer và report generation.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã kiểm tra/bổ sung:

- Code tạo AI triage report.
- Code hoặc output tạo testcase/checklist.
- Các section bắt buộc trong output để phục vụ human validation.

Mục 5 - User Guide Summary Cho Nội Dung Được Giao

1. Prompt + công cụ AI

- Công cụ: GPT-5.5 cho Lê Trung Kiên.
- Prompt/nội dung yêu cầu:

Yêu cầu AI tóm tắt nội dung được giao vào user guide.

2. Nội dung AI hỗ trợ

AI hỗ trợ tóm tắt phần nội dung Trung Kiên phụ trách để đưa vào user guide, gồm các phần liên quan tới DAST/ZAP, scan workflow, AI triage và cách đọc output.

Lê Trung Kiên đã đọc lại, kiểm tra và chỉnh nội dung trước khi giữ trong tài liệu.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

AI hỗ trợ tốt ở tác vụ tóm tắt tài liệu. Nội dung đã được sinh viên kiểm tra lại trước khi sử dụng.

4. Lý do theo ISTQB

- Đây là documentation assistance, không phải evidence kiểm thử độc lập.
- Sinh viên đã human review nội dung tóm tắt.
- Nội dung được đối chiếu với workflow/script/report trong repository.
- Rủi ro chính là thiếu/chệch ý, đã được giảm bằng bước đọc và sửa lại của sinh viên.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã dùng output AI để hỗ trợ:

- Tóm tắt phần hướng dẫn DAST/ZAP.
- Mô tả luồng scan và triage.
- Chỉnh lại nội dung user guide theo phạm vi công việc được giao.

Mục 6 - Khởi Tạo Slide Từ Outline Và User Guide

1. Prompt + công cụ AI

- Công cụ: GPT-5.5 cho Lê Trung Kiên.
- Công cụ/phương pháp hỗ trợ: taste skill để cải thiện giao diện slide.
- Prompt/nội dung yêu cầu:

Yêu cầu AI khởi tạo slide từ outline thuyết trình do Kiên viết và user guide, có sử dụng taste skill để giao diện đẹp hơn.

2. Nội dung AI hỗ trợ

AI hỗ trợ tạo slide deck ban đầu:

- Dựa trên outline thuyết trình do Lê Trung Kiên viết.
- Dựa trên nội dung trong user guide.
- Tổ chức các phần chính của seminar vào slide.
- Áp dụng taste skill để cải thiện bố cục, typography và giao diện trình bày.

Lê Trung Kiên đã kiểm tra lại slide, xác nhận slide đủ và đúng nội dung cần trình bày.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

Slide được AI hỗ trợ khởi tạo từ outline/user guide đã có human review, nội dung được kiểm tra lại về độ đủ và đúng trước khi sử dụng.

4. Lý do theo ISTQB

- Đây là documentation/presentation generation, không phải kết luận defect.
- Input chính đến từ outline và user guide do sinh viên kiểm soát.
- Sinh viên đã review nội dung slide sau khi AI tạo.
- Taste skill chỉ hỗ trợ trình bày giao diện, không thay thế kiểm tra nội dung kỹ thuật.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã kiểm tra:

- Slide có đủ các phần cần thuyết trình.
- Nội dung slide khớp với user guide và outline.
- Bố cục/giao diện đạt yêu cầu trình bày seminar.

Mục 7 - Final Report Generation

1. Prompt + công cụ AI

- Công cụ: GPT-5.5 cho Lê Trung Kiên.
- Prompt/nội dung yêu cầu:

Yêu cầu AI tạo file final-report từ các nội dung đã có trong repository.

2. Nội dung AI hỗ trợ

AI hỗ trợ khởi tạo file final report bằng cách tổng hợp các phần nội dung liên quan trong repository.

Lê Trung Kiên đã đọc lại và sửa:

- Thứ tự section trong báo cáo.
- Path đến các nội dung liên quan.
- Bổ sung link YouTube.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

INCOMPLETE

AI tạo được bản nháp final report, nhưng cần sinh viên đọc, sắp xếp lại section, sửa path và bổ sung link trước khi có thể dùng làm bản nộp cuối.

4. Lý do theo ISTQB

- Đây là report generation có rủi ro sai traceability/path nếu AI tự suy luận từ repository.
- Output ban đầu chưa đủ tin cậy để dùng trực tiếp vì cần chỉnh lại thứ tự section và đường dẫn.
- Sinh viên đã human review và sửa, nhưng trạng thái audit vẫn giữ **INCOMPLETE** để minh bạch rằng AI output gốc chưa đạt chuẩn final evidence.
- Cần kiểm tra thủ công toàn bộ link/path và nội dung tham chiếu trước khi nộp.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã bổ sung/chỉnh sửa:

- Thứ tự section trong final report.
- Path đến các nội dung liên quan.
- Link YouTube cho phần nộp seminar.

Mục 8 - Semgrep Hardcoded JWT Secret: PoC Forge Token

1. Prompt + công cụ AI

- Công cụ theo W05:
 - ChatGPT/Gemini cho Lâm Hữu Khánh.

- Google Gemini 3.1 Flash cho Lê Mai Hoài Bảo.
- Prompt theo weekly report:

Yêu cầu AI hỗ trợ rà soát phần Semgrep SAST, kiểm tra finding `hardcoded-jwt-secret`, giải thích nguy cơ hardcoded JWT secret, gợi ý PoC tạo JWT giả mạo và đề xuất hướng khắc phục bằng biến môi trường.

Yêu cầu AI phân tích finding `hardcoded-jwt-secret` từ kết quả Semgrep và hỗ trợ điền `Track\_A\_Semgrep\_Template.md`.

- Prompt chi tiết lưu tại: [src/semgrep/output/findings/001_javascript-jsonwebtoken-security-jwt-hardcode-hardcoded-jwt-secret_prompt.md](#)

2. Nội dung AI hỗ trợ

AI output gốc lưu tại:

- [src/semgrep/output/findings/001_javascript-jsonwebtoken-security-jwt-hardcode-hardcoded-jwt-secret_ai_output.md](#)
- [docs/semgrep/AI_Triage_hardcoded-jwt-secret.md](#)

Phần PoC/ý tưởng kiểm chứng AI gợi ý:

Mục tiêu: Chứng minh rằng kẻ tấn công có thể sử dụng khóa bí mật bị lộ để tạo JWT giả mạo và xác thực thành công vào hệ thống.

Kịch bản:

1. Xác định khóa bí mật: Truy cập vào mã nguồn của EShop, tìm đến file `eshop-sut/backend/server.js` và lấy giá trị của `SECRET\_KEY` tại dòng 51. Giả sử `SECRET\_KEY` là `"mySuperSecretKey123!"`.
2. Tạo JWT giả mạo với payload `{"id": "999", "role": "admin"}`.
3. Gửi token trong header `Authorization` của request đến API EShop.

Kết quả mong đợi: Nếu ứng dụng không có các biện pháp kiểm tra bổ sung hoặc kiểm tra tính hợp lệ của `id`/`role` trước khi cấp quyền, token giả mạo có thể được xác thực là admin.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

INCOMPLETE

Ý tưởng kiểm chứng đúng, nhưng bản AI ban đầu chưa đủ để dùng làm bằng chứng kiểm thử vì dùng secret giả định, endpoint kiểm chứng còn chung chung, và expected result có điều kiện "nếu..." nên oracle chưa dứt khoát.

4. Lý do theo ISTQB

- Test objective rõ: kiểm tra authentication bypass/privilege escalation do hardcoded JWT secret.
- Test data chưa khớp SUT: AI dùng "mySuperSecretKey123!" thay vì secret thật trong evidence.
- Expected result chưa đo được bằng HTTP status/body cụ thể.
- Theo ISTQB, đây là phần đặc tả kiểm chứng chưa hoàn chỉnh: có ý tưởng kiểm thử, nhưng thiếu precondition, dữ liệu kiểm thử chính xác, bước execute cụ thể và oracle.
- Vì finding liên quan authentication/authorization, nên theo risk-based testing cần ưu tiên xác minh bằng PoC tái lập được.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã sửa PoC bằng evidence thật trong `weekly-reports/Group06_05/evidence/semgrep/finding_hardcoded_jwt_secret.md` và `src/semgrep/exploit.js`:

```
const jwt = require("jsonwebtoken");
const forgedToken = jwt.sign(
  { id: 1, role: "admin" },
  "super_secret_key_that_should_not_be_here"
);
console.log("Bearer " + forgedToken);
```

Bản đã sửa/bổ sung:

- Dùng secret thật: `super_secret_key_that_should_not_be_here`.
- Gộp các finding `server.js:9`, `server.js:51`, `server.js:105` thành 1 root cause.
- Bổ sung remediation: dùng `process.env.JWT_SECRET`, không commit `.env`, rotate token/secret cũ.
- Nên bổ sung thêm request runtime đến endpoint protected/admin với expected `200/403` trước và sau khi fix.

Mục 9 - Semgrep Cleartext HTTP Request: Ý tưởng kiểm chứng bằng proxy

1. Prompt + công cụ AI

- Công cụ theo W06: Codex/ChatGPT, OpenRouter, model Gemini qua OpenRouter `google/gemini-2.5-flash-lite`.
- Prompt theo weekly report:

Yêu cầu AI sửa script để chạy tất cả findings theo số lượng lỗi trong JSON, không hardcode một lỗi.

Yêu cầu AI đọc tài liệu Semgrep trong repo và hướng dẫn cách chạy Semgrep từ bước cài đặt, scan đến AI triage.

- Prompt chi tiết cho finding lưu tại: `src/semgrep/output/findings/004_typescript-react-security-react-insecure-request-react-insecure-request_prompt.md`

2. Nội dung AI hỗ trợ

AI output gốc lưu tại:

- `src/semgrep/output/findings/004_typescript-react-security-react-insecure-request-react-insecure-request_ai_output.md`

Phần hướng dẫn kiểm chứng AI gợi ý:

Yêu cầu công cụ:

- * Một thiết bị chạy ứng dụng EShop hoặc trình giả lập.
- * Một công cụ proxy mạng như Charles Proxy, mitmproxy, Fiddler.
- * Cài đặt chứng chỉ của công cụ proxy trên thiết bị nếu cần xem HTTPS.

Các bước thực hiện:

1. Cấu hình proxy mạng.
2. Bật giám sát HTTP.
3. Chạy ứng dụng EShop và đăng nhập.
4. Truy cập màn hình danh sách đơn hàng.
5. Quan sát traffic trên proxy.
6. Nếu request gửi qua HTTP, kiểm tra header `Authorization`, query/body và response.

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

INCOMPLETE

Hướng kiểm chứng đúng nhưng chưa đủ để kết luận defect trong môi trường seminar. Weekly report chưa lưu giá trị runtime của `API_URL`, log proxy, screenshot request, hoặc expected result cụ thể.

4. Lý do theo ISTQB

- Test objective có liên quan trực tiếp đến CWE-319 và nguy cơ lộ token qua HTTP.
- Precondition thiếu: chưa rõ build nào, `API_URL` nào, thiết bị/proxy nào, tài khoản test nào.
- Expected result chưa thành oracle: cần nêu rõ request phải dùng HTTPS hay HTTP, header nào được quan sát, response nào được coi là fail.
- Có nguy cơ false positive/environment noise vì local/dev có thể dùng HTTP có chủ đích.
- Cần bằng chứng confirmation testing: log proxy hoặc screenshot request/response.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã bổ sung:

- `src/semgrep/output/semgrep_triage_report.md`: ghi 12 findings và để **Human Validation = Needs Manual Verification**.
- `weekly-reports/Group06_05/evidence/semgrep/failure_modes_metrics.md`: ghi rõ cần kết hợp DAST/ZAP và human audit.

Nội dung kiểm chứng nên chốt khi nộp:

Verify mobile order API does not transmit Authorization token over cleartext HTTP
Precondition: EShop backend/frontend-mobile chạy với cấu hình demo; user test đã đăng nhập và có đơn hàng.

Steps:

1. Cấu hình emulator/device đi qua mitmproxy hoặc ZAP proxy.
2. Đăng nhập user test.
3. Mở màn hình My Orders.
4. Lọc request đến `/orders/my-orders``.

Expected:

- Với môi trường production/demo, request phải dùng HTTPS.
- Nếu request dùng HTTP và header ``Authorization: Bearer ...`` đọc được trên proxy thì mark Confirmed.

Evidence:

- Lưu proxy screenshot/log request + response header.

Mục 10 - Semgrep AI Triage Workflow: Unit Tests Cho All Findings

1. Prompt + công cụ AI

- Công cụ theo W06: Codex/ChatGPT, OpenRouter, model Gemini qua OpenRouter `google/gemini-2.5-flash-lite`.
- Prompt theo weekly report:

Yêu cầu AI sửa cấu hình ``.env.example`` để dùng OpenRouter với model Gemini thay vì dùng trực tiếp ``GEMINI_API_KEY``.

Yêu cầu AI sửa script để đọc ``OPENROUTER_API_KEY`/`OPENROUTER_BASE_URL``.

Yêu cầu AI sửa script để chạy tất cả findings theo số lượng lỗi trong JSON, không hardcoded một lỗi.

2. Nội dung AI hỗ trợ

Nội dung liên quan được audit trong `src/semgrep/test_semgrep_ai_triage.py`:

```

def test_collect_finding_records_keeps_all_findings_and_metadata(self):
    ...
    self.assertEqual(len(records), 2)
    self.assertEqual(records[0].rule_id, "rule.one")
    self.assertEqual(records[0].cwe, "CWE-798")
    self.assertEqual(records[0].owasp, "A07:2025")
    self.assertIn("SECRET_KEY", records[0].code)
    self.assertEqual(records[1].severity, "ERROR")

def test_get_ai_settings_supports_openrouter_variables(self):
    ...
    self.assertEqual(settings.provider, "openai-compatible")
    self.assertEqual(settings.model, "google/gemini-2.5-flash-lite")
    self.assertEqual(settings.api_key, "openrouter-key")
    self.assertEqual(settings.base_url, "https://openrouter.ai/api/v1")

def test_write_triage_outputs_creates_summary_and_per_finding_files(self):
    ...
    self.assertIn("semgrep_triage_report", report_path.name)
    self.assertIn("Human Validation", report)
    self.assertIn("const SECRET_KEY", prompt)
    self.assertIn("AI output body", ai_output)

```

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

Nhóm unit tests này hợp lệ ở mức component/unit testing. Test không gọi API thật, có fixture tối thiểu, có assertion rõ ràng và kiểm tra đúng regression quan trọng: không hardcode một finding, giữ metadata, tạo output prompt/AI-output riêng.

4. Lý do theo ISTQB

- Test level phù hợp: component test cho parser/config/output writer.
- Test độc lập với external services, credentials và network.
- Có expected result cụ thể bằng assertions.
- Có regression value cao cho lỗi W06: script phải xử lý đủ số lượng findings trong JSON.
- Giới hạn còn lại: chưa phải system test với OpenRouter live, nhưng điều này chấp nhận được cho unit test.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã bổ sung:

- `src/semgrep/test_semgrep_ai_triage.py`
- `src/semgrep/output/semgrep_triage_report.md`
- `src/semgrep/output/findings/*_prompt.md`

- `src/semgrep/output/findings/*_ai_output.md`

Kết quả kỳ vọng của bản đã sửa:

- Input có 12 findings thì report ghi `Total findings in input: 12`.
- Mỗi finding có prompt và AI output riêng.
- Finding chưa được human validate thì giữ trạng thái `Needs Manual Verification`, không tự động kết luận confirmed.

Mục 11 - ZAP CLI Scan Mode, Report Format, Output Path

1. Prompt + công cụ AI

- Công cụ theo W06: Codex, model GPT-5.4 và GPT-5.5.
- Prompt theo weekly report:

Yêu cầu AI thêm cờ để ZAP chạy theo ``basic`` hoặc ``owasp-top10-2025``, nhưng vẫn giữ nguyên nguyên tắc ZAP scan toàn bộ và chỉ lọc active policy theo tag/rule tương ứng khi cần.

Yêu cầu AI thêm tùy chọn cho người dùng tự chọn path và tên file output, phục vụ tách report cho frontend user, frontend admin hoặc backend.

Yêu cầu AI đổi output report từ chỉ HTML sang cho phép xuất JSON để pipeline sau này có thể đọc bằng AI và đối chiếu với PoC từ Semgrep.

2. Nội dung AI hỗ trợ

Nội dung liên quan được audit trong `src/zap/test_scan_zap.py`:

```
def test_output_file_alias_selects_report_path(self):
    args = build_parser(load_env=False).parse_args(
        ["--output-file", "src/zap/output/frontend_user.html"]
    )
    self.assertEqual(args.report_file, "src/zap/output/frontend_user.html")

def test_parser_rejects_non_pipeline_report_formats(self):
    for report_format in ("xml", "md"):
        with self.subTest(report_format=report_format):
            with self.assertRaises(SystemExit):
                build_parser(load_env=False).parse_args(
                    ["--report-format", report_format]
                )

def
```

```
test_configure_scan_policy_creates_owasp_top10_2025_policy_from_alert_tags(self):
    ...
    zap.ascan.enable_scanners.assert_called_once_with(
        "40012,40018",
        scanpolicyname=OWASP_TOP10_2025_POLICY_NAME,
    )
```

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

Đây là nhóm regression tests hợp lệ cho CLI behavior và scan policy configuration. Test có oracle rõ, dùng mock ZAP API, không cần live daemon, và trace được tới yêu cầu W06 về `basic`, `owasp-top10-2025`, `html/json`, `--output-file`.

4. Lý do theo ISTQB

- Có traceability giữa requirement và test.
- Có negative testing: reject `xml`, `md`; reject trường hợp không có scanner OWASP matching.
- Có robustness testing: fallback khi ZAP API không trả alert tags.
- Mock giúp test nhanh, ổn định, không phụ thuộc ZAP daemon.
- Cần thêm smoke/system test với ZAP daemon thật để xác nhận official template có sẵn trong image.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã bổ sung:

- `src/zap/test_scan_zap.py`
- `src/zap/scan_zap.py`
- `src/zap/README.md`

Bản đã sửa:

- `--report-format` chỉ nhận `html` hoặc `json`.
- `--output-file` là alias của `--report-file`.
- `basic` giữ default enabled rules.
- `owasp-top10-2025` lọc active scanner theo tag `OWASP_2025_A*`, có fallback theo rule ID nếu ZAP API không trả tag.

Mục 12 - ZAP Authenticated Scan: JWT, Context, Replacer, Cleanup

1. Prompt + công cụ AI

- Công cụ theo W06: Google Gemini, model Gemini 3.1 Pro.
- Prompt theo weekly report:

Yêu cầu AI sinh mã cấu hình authenticated scan cho ZAP trên EShop, bao gồm đăng nhập lấy JWT, cấu hình context, user, forced-user và gắn `Authorization: Bearer ...` khi scan.

Yêu cầu AI gợi ý cách dùng AJAX Spider cho frontend SPA và cách giới hạn scope chỉ trong các URL local của EShop.

Yêu cầu AI giải thích vì sao login qua ZAP có thể trả `401` hoặc `403`, sau đó đối chiếu lại với tài khoản test và cơ chế proxy của ZAP.

2. Nội dung AI hỗ trợ

Nội dung liên quan được audit trong `src/zap/test_zap_runtime.py` và `src/zap/test_scan_zap.py`:

```
def test_validate_target_accepts_eshop_local_ports(self):
    for url in (
        "http://localhost:3000",
        "http://localhost:5173/",
        "http://127.0.0.1:5174/admin",
    ):
        with self.subTest(url=url):
            self.assertEqual(validate_target(url), url)

def test_validate_target_rejects_external_or_unknown_port(self):
    for url in ("https://example.com", "http://localhost:8080"):
        with self.subTest(url=url):
            with self.assertRaisesRegex(ValueError, "local EShop"):
                validate_target(url)

def test_configure_authenticated_context_creates_scoped_user_and_replacer(self):
    ...
    zap.replacer.add_rule.assert_called_once_with(
        REPLACER_RULE,
        "true",
        "REQ_HEADER",
        "false",
        "Authorization",
        "Bearer jwt-value",
        url=CONTEXT_REGEX,
    )

def test_cleanup_removes_secret_and_disables_forced_user(self):
    cleanup_authenticated_context(zap, forced_user=True)
    zap.replacer.remove_rule.assert_called_once_with(REPLACER_RULE)
    zap.forcedUser.set_forced_user_mode_enabled.assert_called_once_with("false")
```

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

VALID

Nhóm unit tests hợp lệ cho component/integration-with-mocks. Test kiểm tra các đường an toàn quan trọng: chỉ scan target local, không đẩy credential qua remote ZAP, inject JWT qua Replacer, bật/tắt Forced User Mode và cleanup secret.

4. Lý do theo ISTQB

- Test objective gắn với security control: authenticated scan phải đúng context/scope và không làm lộ credential.
- Có negative testing cho URL ngoài, malformed port, remote ZAP URL.
- Có oracle cụ thể trên API call của ZAP mock.
- Có test cleanup, quan trọng vì JWT là sensitive test data.
- Cần manual/live verification riêng cho lỗi 401/403 và coverage AJAX Spider.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã bổ sung:

- `src/zap/zap_runtime.py`
- `src/zap/test_zap_runtime.py`
- `src/zap/test_scan_zap.py`
- `src/zap/README.md`

Bản đã sửa:

- `validate_target()` chỉ cho `localhost/127.0.0.1` trên port `3000`, `5173`, `5174`.
- Authenticated scan chỉ chấp nhận ZAP daemon local HTTP.
- `login_for_token()` đi qua ZAP proxy và không in JWT ra stdout/stderr.
- `cleanup_authenticated_context()` xóa Replacer rule và tắt Forced User Mode.

Mục 13 - ZAP JSON -> OpenRouter: PoC Extraction Và AI Report

1. Prompt + công cụ AI

- Công cụ: Codex/OpenRouter, model `google/gemini-2.5-flash`.
- Prompt trong design/source:

```
Create a CLI script that reads one or more OWASP ZAP JSON reports from
`src/zap/output`, sends the parsed alert instances to OpenRouter using a Gemini
model, and writes a user-selected Markdown or HTML report.
```

- Evidence:
 - `src/zap/test_zap_ai_triage.py`

- `src/zap/output/zap_ai_triage_report.md`

2. Nội dung AI hỗ trợ

Nội dung liên quan được audit trong `src/zap/test_zap_ai_triage.py`:

```
def test_parse_multiple_json_files_extracts_every_alert_instance_and_tags(self):
    ...
    alerts = parse_zap_json_files([first, second])
    self.assertEqual(len(alerts), 3)
    self.assertEqual(alerts[0].alert_name, "SQL Injection")
    self.assertEqual(alerts[0].risk, "High")
    self.assertEqual(alerts[0].owasp_tags, ["OWASP_2021_A03"])
    self.assertEqual(alerts[0].poc["method"], "GET")
    self.assertEqual(alerts[0].poc["payload"], "' OR '1'='1'")

def test_build_prompt_contains_required_sections_for_ai(self):
    ...
    self.assertIn("Chi tiết + giải thích lỗi", prompt)
    self.assertIn("Tag OWASP", prompt)
    self.assertIn("PoC", prompt)
    self.assertIn("Cách verify PoC", prompt)
    self.assertIn("OWASP_2021_A01", prompt)
    self.assertNotIn("CWE-264", prompt)
```

AI output report có ví dụ verify PoC trong `src/zap/output/zap_ai_triage_report.md`:

```
PoC (Proof of Concept):
- Method: `GET`
- Endpoint: `http://localhost:3000`
- Payload: (Không có payload cụ thể)
- Notes: Thực hiện lại yêu cầu GET và kiểm tra các header/body phản hồi.

Cách verify PoC:
1. Mở Postman hoặc công cụ tương tự.
2. Tạo một yêu cầu `GET` đến `http://localhost:3000`.
3. Gửi yêu cầu.
4. Expected: Trong phần header phản hồi, tìm kiếm header `Access-Control-Allow-Origin`. Giá trị của nó là `*`.
5. Actual: Header `Access-Control-Allow-Origin: *` được tìm thấy.
6. Header/Body cần kiểm tra: `Access-Control-Allow-Origin` trong response header.
```

Screenshot: không có screenshot trực tiếp của AI output trong weekly report.

3. Verdict

INCOMPLETE

Unit tests cho parser/prompt/rendering là tốt, nhưng AI-generated report vẫn cần audit lại trước khi dùng làm evidence cuối. Output có tag **OWASP_22021_A01** sai chính tả, nhiều CSP alerts bị lặp root cause, và phần "Actual" được ghi như đã verify nhưng không kèm log/screenshot Postman.

4. Lý do theo ISTQB

- Parser unit tests có expected result rõ, nhưng report AI là test procedure/bug report nên cần evidence execute riêng.
- Theo ISTQB, actual result chỉ được ghi sau khi đã execute test.
- Cần traceability từ ZAP JSON instance -> PoC -> request/response log -> observed header.
- Cần defect deduplication: nhiều CSP alerts trên nhiều URL có thể là một root cause header policy.
- Sai taxonomy/tag OWASP là lỗi chất lượng dữ liệu trong output AI.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã bổ sung:

- `src/zap/test_zap_ai_triage.py`
- `src/zap/zap_ai_triage.py`
- `src/zap/output/zap_ai_triage_report.md`

Nội dung kiểm chứng nên sửa trước khi nộp:

```
Verify CORS wildcard header on backend
Precondition: backend EShop running at http://localhost:3000; ZAP/Postman
available.
Steps:
1. Send GET http://localhost:3000 with Origin: http://evil.example.
2. Inspect response header.
Expected:
- If Access-Control-Allow-Origin is `*`, finding is Confirmed only when endpoint
can expose sensitive data or credentials are allowed in a risky way.
- If no wildcard header or endpoint is public/static only, mark Needs Manual
Verification or Environment Noise.
Evidence:
- Attach raw request/response headers or screenshot.
```

Mục 14 - Semgrep AI Triage Refinement Trong Phiên Chat: Source Context, Fail-Fast, Report Format

1. Prompt + công cụ AI

- Công cụ: Codex/ChatGPT trong phiên làm việc trực tiếp với repository.
- Các yêu cầu chính từ phiên chat:

Sửa context là đọc source sau đó phân loại true/false/need human review.

Nếu gọi AI bị lỗi thì cho nó fail hoặc stop chứ không cho gen ra các finding.

Hãy xem xét các output của AI là tiếng Việt nhé.

Hãy bỏ script tạo postman validation, thêm script tạo bảng test case có include tới finding nào.

Test case làm theo từng entry chứ không sinh ra bảng.

Mỗi entry trong semgrep_triage_report thêm tag lỗi CWE/OWASP.

Format lại semgrep_triage_report để heading AI kiểu ## Triage Finding không làm mất cân đối.

Bảng trong semgrep_triage_report dùng tiếng Việt, trừ tên tag lỗi.

2. Nội dung AI hỗ trợ

AI đã hỗ trợ chỉnh workflow Semgrep trong `src/semgrep/semgrep_ai_triage.py` và tài liệu `submission/User_Guide.md`.

Các thay đổi kỹ thuật chính:

- Prompt AI yêu cầu đọc source evidence trước khi phân loại.
- Chỉ dùng ba trạng thái triage: `True Positive`, `False Positive`, `Needs Human Review`.
- AI output yêu cầu viết bằng tiếng Việt, trừ thuật ngữ chuẩn như `CWE`, `OWASP`, `True Positive`.
- Thêm giới hạn `AI_MAX_TOKENS` để tránh lỗi request quá lớn qua OpenRouter.
- Nếu AI provider lỗi, ví dụ OpenRouter `402`, script dừng với exit code `1` và không sinh report thiếu phân tích.
- Bỏ report Postman validation cũ, thay bằng `semgrep_test_cases.md` dạng từng entry test case.
- `semgrep_triage_report.md` có tag lỗi trong từng finding entry: `Rule`, `Severity`, `CWE`, `OWASP`, `Likelihood`, `Impact`, `Confidence`.
- Heading do AI sinh ra khi nhúng inline vào report được hạ cấp để không phá hierarchy Markdown.
- Bảng tổng hợp trong `semgrep_triage_report.md` dùng tiếng Việt: `Quy tắc`, `Tệp`, `Mức độ`, `Kết quả AI`, `Trạng thái kiểm chứng`.

Các regression tests liên quan trong `src/semgrep/test_semgrep_ai_triage.py`:

```
def
test_build_prompt_includes_source_first_three_state_classification_context(self):
    ...
    self.assertIn("Đọc và đối chiếu source evidence trước khi phân loại", prompt)
    self.assertIn("Phân loại: True Positive / False Positive / Needs Human
Review", prompt)
    self.assertNotIn("PoC hoặc testcase kiểm chứng", prompt)

def test_main_stops_without_writing_reports_when_ai_call_fails(self):
    ...
    self.assertEqual(exit_code, 1)
    write_mock.assert_not_called()

def test_write_triage_outputs_creates_test_case_entries_report(self):
    ...
    self.assertIn("## TC-SEMGREP-001", report)
    self.assertIn("### Input", report)
    self.assertIn("### Thao tác", report)
    self.assertNotIn("### Expected output", report)
    self.assertNotIn("### Actual output", report)

def test_triage_report_detail_entries_include_security_tags(self):
    ...
    self.assertIn("#### Tags lỗi", report)
    self.assertIn("`CWE-798: Use of Hard-coded Credentials`", report)
    self.assertIn("`A07:2021 - Identification and Authentication Failures`",
report)

def test_triage_report_demotes_ai_headings_inside_finding_details(self):
    ...
    self.assertNotIn("\n## Triage Finding Bảo Mật SEMGREP-001", report)
    self.assertIn("\n##### Triage Finding Bảo Mật SEMGREP-001", report)
```

Verification đã chạy trong phiên:

```
python3 -m unittest discover -s src/semgrep -p 'test_*.py'
```

Kết quả cuối cùng: `Ran 15 tests ... OK.`

3. Verdict

VALID

Các chỉnh sửa này hợp lệ ở mức workflow/component regression vì có test tự động bảo vệ những yêu cầu chính: source-first context, phân loại ba trạng thái, fail-fast khi AI lỗi, không sinh Postman validation report cũ, test case dạng entry, tag CWE/OWASP trong report và format Markdown không bị lệch heading.

4. Lý do theo ISTQB

- Traceability rõ từ yêu cầu trong phiên chat đến test case tự động.
- Có negative/regression testing cho lỗi provider: script không sinh report khi AI lỗi.
- Có test oracle cụ thể bằng assertion trên output Markdown.
- Giảm nguy cơ false evidence: AI không tự sinh "Actual result" hoặc finding khi provider lỗi.
- Tách vai trò hợp lý: AI phân tích finding, còn test case kiểm chứng nằm ở file riêng để tester thực thi và điền kết quả.
- Giới hạn còn lại: regression tests xác minh format/script, chưa thay thế manual validation bằng runtime request, source review và evidence thật.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên đã bổ sung/cập nhật:

- `src/semgrep/semgrep_ai_triage.py`
- `src/semgrep/test_semgrep_ai_triage.py`
- `submission/User_Guide.md`

Kết quả workflow hiện tại:

- `semgrep_triage_report.md`: báo cáo tổng hợp all findings, có bảng tiếng Việt, tag lỗi, source evidence và AI analysis đã normalize heading.
- `semgrep_test_cases.md`: danh sách test case theo từng entry, có **Finding liên quan**, **Input**, **Thao tác**, **Kết quả cần ghi nhận**, **Trạng thái**.
- `findings/*_prompt.md`: prompt từng finding, yêu cầu AI đọc source và phân loại ba trạng thái.
- `findings/*_ai_output.md`: raw output từng finding, giữ nguyên để đối chiếu.

Mục 15 - Format Semgrep Finding Thành Test Case Postman

1. Prompt + công cụ AI

- Công cụ: Codex/ChatGPT trong phiên làm việc trực tiếp với repository.
- Prompt hoàn thiện:

Hãy đề xuất format chuẩn để chuyển finding Semgrep thành test case có thể kiểm chứng bằng Postman. Format cần gần giống alert bảo mật nhưng tách rõ `Source Evidence` và `Runtime Mapping`, vì Semgrep là SAST nên không tự có endpoint/request/response. Mỗi finding cần có Rule ID, severity, CWE, OWASP, file, line, vulnerable code, method, URL, headers, payload mẫu, pre-test setup, expected vulnerable behavior, expected secure behavior, status kiểm chứng và mapping confidence.

2. Nội dung AI hỗ trợ

AI hỗ trợ thiết kế format Markdown cho từng finding Semgrep:

- **Alert Summary**: tool, type, rule, severity, CWE, OWASP, status.

- **Source Evidence:** file, line, vulnerable code.
- **Runtime Mapping:** affected feature, method, endpoint, base URL, auth, confidence.
- **Postman Test Case:** URL, headers, payload, pre-test setup.
- **Expected Result:** vulnerable behavior, secure behavior, evidence cần thu thập.
- **Human Validation:** trạng thái cần tester cập nhật sau khi chạy PoC.

Format này được dùng làm nền để cập nhật `src/semgrep/semgrep_ai_triage.py`, `semgrep_triage_report.md` và `semgrep_test_cases.md`.

3. Verdict

VALID

4. Lý do theo ISTQB

- Test objective rõ: biến static finding thành manual test case có thể execute.
- Có precondition, input, action và expected result.
- Có phân biệt source evidence và runtime evidence, giảm rủi ro kết luận sai từ SAST.
- Có trường confidence/status để minh bạch finding cần human validation.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên yêu cầu:

- Đồng bộ output tiếng Việt, trừ thuật ngữ chuẩn.
- Không sinh bảng test case quá khó đọc, mà chuyển sang từng entry riêng.
- Không ghi **Actual result** nếu chưa chạy Postman thật.
- Bổ sung payload mẫu để tester có thể thay dữ liệu và chạy PoC.

Mục 16 - Payload Mẫu Tự Động Cho Postman/PoC Từ Semgrep Finding

1. Prompt + công cụ AI

- Công cụ: Codex/ChatGPT trong phiên làm việc trực tiếp với repository.
- Prompt hoàn thiện:

Hãy bổ sung khả năng tự sinh payload mẫu cho test case Postman từ source code Semgrep finding. Với request có ``body: JSON.stringify(...)``, script cần trích field trong object và map sang giá trị mẫu hoặc biến Postman như ``{{test_email}}``, ``{{test_password}}``, ``{{coupon_code}}``. Với GET/HEAD thì ghi rõ không có request body. Payload chỉ là mẫu để tester thay dữ liệu thật khi chạy PoC, không được ghi như actual result.

2. Nội dung AI hỗ trợ

AI hỗ trợ thêm logic trong `src/semgrep/semgrep_ai_triage.py`:

- Parse `body: JSON.stringify(...)` từ source snippet.

- Tách các field top-level trong object request body.
- Sinh payload JSON mẫu cho Postman.
- Sinh giá trị mẫu cho field phổ biến như `email`, `password`, `name`, `phone`, `items`, `coupon_id`.
- Với `GET/HEAD`, ghi rõ **Không có request body**.
- Ưu tiên endpoint/payload gần dòng finding được Semgrep đánh dấu.
- Nhúng block `Postman/PoC` tự động vào từng finding trong `semgrep_triage_report.md`.

3. Verdict

VALID

4. Lý do theo ISTQB

- Test data có traceability từ source code, không phải AI tự bịa hoàn toàn.
- Có phân biệt payload mẫu với actual runtime result.
- Có negative path: nếu không trích được body thì payload `{}` và confidence thấp.
- Có unit tests cho login payload, checkout payload, GET no body và marked finding line.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên yêu cầu generator tự sinh payload trong output mới, không chỉnh trực tiếp file generated. Sau đó script và tests được cập nhật để khi chạy lại AI triage, payload mẫu được sinh tự động trong `semgrep_triage_report.md` và `semgrep_test_cases.md`.

Mục 17 - User Guide Và Debug Cài Đặt Semgrep Trên Windows

1. Prompt + công cụ AI

- Công cụ: Codex/ChatGPT trong phiên làm việc trực tiếp với repository.
- Prompt hoàn thiện:

Hãy cập nhật user guide Semgrep để người dùng chạy được flow thủ công trên Windows, macOS và Linux. Guide cần có bước tạo venv, cài dependency triage, cài Semgrep CLI, scan source `./eshop-sut`` hoặc dùng ``SOURCE_ROOT`` nếu source nằm nơi khác, chạy AI triage, kiểm tra output và xử lý lỗi thường gặp như PowerShell không dùng ``source``, command xuống dòng bằng ``\``, path WSL/Windows không tương thích, Unicode ``charmap``, ``python` not found`, ``failed to locate pyenv.cfg``, lỗi ``maturin/rpds-py/Rust not found`` và lỗi provider AI.

2. Nội dung AI hỗ trợ

AI hỗ trợ cập nhật `submission/User_Guide.md` và giải thích lỗi môi trường:

- `source .venv/bin/activate` không dùng trong PowerShell; Windows dùng `.\.venv\Scripts\Activate.ps1`.
- PowerShell không dùng dấu `\` để xuống dòng như Git Bash.
- Path `/mnt/d/...` là WSL path; Semgrep Windows cần path Windows hoặc chạy trong cùng môi trường shell.

- `python` not found ở Bash có thể do PATH khác PowerShell.
- `failed to locate pyvenv.cfg` thường do `.venv` hỏng hoặc Python/PATH trỏ nhầm.
- Lỗi `maturin/rpds-py/Rust not found` là lỗi build dependency native trong pip environment, không phải lỗi source EShop.
- Theo trạng thái cuối, guide cài Semgrep CLI bằng `python -m pip install semgrep` và kiểm tra bằng `semgrep --version`.

3. Verdict

VALID

4. Lý do theo ISTQB

- Đây là environment/setup documentation, không phải evidence defect của SUT.
- Các lỗi được phân tích từ log thực tế của sinh viên.
- Có command kiểm tra và expected result rõ ở từng bước.
- Có phân biệt lỗi tool/environment với lỗi trong ứng dụng EShop.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên yêu cầu:

- Bỏ pipeline tự động, chỉ giữ flow chạy thủ công.
- Đưa option Windows/Git Bash vào từng bước, không dồn hết xuống troubleshooting.
- Bỏ `pipx` khỏi guide khi cài Semgrep bằng pip đã hoạt động.
- Giải thích rõ phần cài Semgrep CLI trong script quay video.

Mục 18 - Kiểm Chứng Output Semgrep AI Triage Sau Khi Generate

1. Prompt + công cụ AI

- Công cụ: Codex/ChatGPT trong phiên làm việc trực tiếp với repository.
- Prompt hoàn thiện:

Hãy đọc các file output được sinh sau khi chạy Semgrep scan và AI triage, gồm ``semgrep_results.json``, ``semgrep_triage_report.md``, ``semgrep_test_cases.md`` và các file trong ``output/findings``. Kiểm tra xem report có đúng số finding, đúng source evidence, đồng bộ tiếng Việt, có tag lỗi, có Postman/PoC tự động, có payload mẫu, có status kiểm chứng và không ghi actual result khi chưa execute hay không. Nêu các điểm sai hoặc cần chỉnh trước khi dùng cho báo cáo cuối và test Postman.

2. Nội dung AI hỗ trợ

AI hỗ trợ review generated artifacts:

- Xác định `semgrep_test_cases.md` đã có payload mẫu nhưng `semgrep_triage_report.md` chưa nhúng block Postman/PoC trong từng finding.
- Đề xuất đưa `Postman/PoC tự động` vào chi tiết finding để reviewer đọc một file vẫn thấy request test.

- Xác định payload nào cần body và payload nào không cần body.
- Nhấn mạnh payload mẫu không phải actual value; tester phải thay bằng data thật.
- Nhấn mạnh **Confirmed** chỉ được ghi sau khi chạy Postman và có evidence.

3. Verdict

VALID

4. Lý do theo ISTQB

- Có artifact review sau khi generate output.
- Có traceability từ Semgrep finding sang test case Postman.
- Có oracle rõ: không được ghi actual result khi chưa execute.
- Có regression test sau khi chỉnh generator để tránh mất block Postman/PoC.

5. Nội dung đã được SV sửa hoặc bổ sung

Sinh viên yêu cầu không sửa trực tiếp output generated, mà phải sửa generator để lần chạy sau tự sinh đúng payload/testcase. Script và test sau đó được cập nhật, verification đã chạy:

```
python -m unittest src.semgrep.test_semgrep_ai_triage
Ran 23 tests ... OK
```

3. Tổng Kết Độ Chính Xác AI

ID	Nhóm test	Verdict	Ghi chú chính
Mục 1	DAST/ZAP/OWASP Top 10 research	VALID	AI hỗ trợ tìm hiểu, SV đối chiếu lại với nguồn chính thống OWASP/ZAP.
Mục 2	ZAP scan output JSON/HTML	VALID	Script output format được SV kiểm tra và bổ sung tag.
Mục 3	ZAP GUI authenticated scan setup	VALID	Hướng dẫn Firefox localhost được SV kiểm tra trong môi trường local.
Mục 4	ZAP AI triage/testcase generation	VALID	Code và output có đủ section yêu cầu, đã được SV kiểm thử và xác nhận.
Mục 5	User guide summary	VALID	AI tóm tắt nội dung được giao, SV đọc lại và chỉnh trước khi dùng.
Mục 6	Slide generation from outline/user guide	VALID	Slide được khởi tạo từ outline/user guide và kiểm tra lại nội dung.
Mục 7	Final report generation	INCOMPLETE	AI tạo bản nháp nhưng cần SV sửa section, path và bổ sung link YouTube.
Mục 8	Semgrep hardcoded JWT PoC	INCOMPLETE	Ý tưởng đúng nhưng AI dùng placeholder secret/endpoint; SV đã sửa bằng exploit.js .

ID	Nhóm test	Verdict	Ghi chú chính
Mục 9	Semgrep cleartext HTTP proxy verification	INCOMPLETE	Cần runtime config, proxy log và oracle rõ.
Mục 10	Semgrep all-findings/unit workflow	VALID	Unit tests deterministic, có oracle, không phụ thuộc API.
Mục 11	ZAP CLI scan mode/report format	VALID	Có positive/negative tests và traceability với yêu cầu W06.
Mục 12	ZAP authenticated scan	VALID	Kiểm tra target allowlist, JWT Replacer, Forced User Mode, cleanup.
Mục 13	ZAP JSON/OpenRouter report	INCOMPLETE	Parser tests tốt nhưng AI report cần dedup, sửa tag sai và thêm evidence execute.
Mục 14	Semgrep AI triage refinement trong phiên chat	VALID	Có regression tests cho source-first prompt, fail-fast AI, report tiếng Việt, test case entry, tag CWE/OWASP và format heading.
Mục 15	Semgrep finding -> Postman testcase format	VALID	Có source evidence, runtime mapping, expected behavior và validation status.
Mục 16	Semgrep payload mẫu tự động	VALID	Payload sinh từ source <code>JSON.stringify</code> , không ghi actual result khi chưa execute.
Mục 17	Semgrep user guide/debug Windows	VALID	Dựa trên lỗi môi trường thực tế và command đã chỉnh trong guide.
Mục 18	Kiểm chứng output Semgrep AI triage	VALID	Review generated artifacts và sửa generator thay vì sửa output thủ công.

Tổng cộng có 18 nhóm nội dung AI được audit:

- 14/18 mục được đánh giá **VALID** ở mức research/documentation support, unit/component/regression testing hoặc workflow có review rõ ràng.
- 4/18 mục được đánh giá **INCOMPLETE** vì còn thiếu runtime evidence, precondition, test data đúng SUT, request/response log, screenshot xác minh hoặc cần sửa bản nháp AI trước khi dùng.
- Tỷ lệ nội dung có thể dùng trực tiếp sau human review: khoảng 77,8%.
- Tỷ lệ nội dung cần bổ sung bằng chứng hoặc chỉnh sửa trước khi dùng làm kết luận cuối: khoảng 22,2%.

AI hữu ích nhất khi hỗ trợ cấu trúc hóa script, test, prompt và report. AI kém tin cậy hơn khi phải tự suy ra dữ liệu runtime, endpoint thật, taxonomy OWASP hoặc trạng thái "Actual" nếu nhóm chưa cung cấp bằng chứng execute.

4. Kết Luận

AI hỗ trợ tốt nhất ở các việc:

- Hỗ trợ rà soát parser, CLI, config và luồng AI triage.
- Gợi ý PoC và remediation cho hardcoded JWT secret.
- Tạo prompt/report format cho AI triage.
- Chuẩn hóa Semgrep finding thành Postman testcase.
- Hỗ trợ tìm hiểu nguồn chính thống, tóm tắt user guide, khởi tạo slide và tạo bản nháp final report.

Các lỗi hoặc hạn chế đã audit được:

- Một số PoC thiếu precondition, test data hoặc expected result.
- Có placeholder thay cho dữ liệu thật của SUT.
- Có nguy cơ ghi "Actual" khi chưa có bằng chứng execution.
- Có duplicate findings cùng root cause.
- Có sai sót taxonomy/tag OWASP trong output AI.
- Có nguy cơ format report bị lệch nếu nhúng raw AI Markdown trực tiếp; đã giảm rủi ro bằng cách normalize heading trong `semgrep_triage_report.md`.
- Có nguy cơ report thiếu phân tích khi provider lỗi; đã giảm rủi ro bằng fail-fast thay vì sinh output lỗi như finding hợp lệ.
- Có nguy cơ dùng payload mẫu như dữ liệu thật; đã giảm rủi ro bằng cách ghi rõ payload chỉ là template và cần tester thay bằng dữ liệu Postman.

Kết luận về mức độ tin cậy: các unit tests đã được sinh viên sửa/bổ sung trong `src/semgrep` và `src/zap` có thể xem là hợp lệ ở mức component/regression testing. Các PoC, hướng kiểm chứng và security report có AI hỗ trợ cần tiếp tục human validation bằng source evidence, request/response log, screenshot hoặc report ZAP/Semgrep trước khi dùng làm bằng chứng chính thức.

5. Disclosure

Nhóm có sử dụng các công cụ AI như ChatGPT/Codex, Google Gemini và Gemini qua OpenRouter để hỗ trợ học công cụ, viết nháp script, tạo prompt triage, giải thích finding, đề xuất PoC, đề xuất remediation và rà soát nội dung báo cáo.

Nhóm không xem AI là nguồn bằng chứng cuối cùng. Các kết luận trong báo cáo chỉ được giữ lại khi đã được thành viên phụ trách đọc lại, chỉnh sửa và đối chiếu với ít nhất một trong các nguồn sau:

- Source code EShop hoặc script trong repository.
- Output Semgrep/ZAP đã lưu.
- Unit test hoặc mock test trong `src/semgrep` và `src/zap`.
- PoC/testcase, request/response log, screenshot hoặc evidence trong weekly reports.
- Tài liệu chính thức của Semgrep, OWASP ZAP, OWASP Top 10 hoặc provider liên quan.

Các phần còn **INCOMPLETE** được giữ nguyên trạng thái để minh bạch rằng AI output hoặc report chưa đủ bằng chứng kiểm thử cuối cùng. Nhóm không sử dụng AI để tạo giả feedback, attendance, log chạy tool, screenshot hoặc kết quả scan.

Chi tiết công bố theo từng thành viên được ghi trong `ai-audit/[AI-03]_AI_Disclosure_Template.md` và cần được ký/xuất thành `ai-audit/[AI-03]_AI_Disclosure.pdf` trước khi nộp.